What Limits Statevector Quantum-Circuit Simulation on Microcontrollers? Memory, Time and Offload on Uno, ESP8266 and ESP32

Anonymous submission

***Abstract*—Can a microcontroller simulate a quantum circuit itself, or should it offload the circuit to a PC? We benchmark a float32 statevector simulator on an Arduino Uno R3, an ESP8266 and an ESP32 for four circuits (`h_all`, `bell_ghz`, `qft`, `grover`), measuring memory limits, execution time and a serial offload to Qiskit Aer. For the first three circuits the memory wall lies at $n = 8$, 13 and 14 qubits and is consistent with $\lfloor \log_2(B/8) \rfloor$, where B is the free memory read at $n = 2$. A per-gate-kind time model fitted at small n predicts held-out run times with 17 % mean absolute percentage error, against 46 % for one constant per board. Offloading overtakes local execution at between $n = 2$ and $n = 13$ qubits, depending on board, circuit and PC speed. The study uses one unit per board, one PC and one baud rate, and several analysis rules were set after the data were seen.**

***Index Terms*—statevector simulation, microcontrollers, memory wall, performance model, computation offloading**

I. Introduction

A dense statevector simulation of n qubits stores 2^n complex amplitudes [1]. On a microcontroller the state competes with the program and the runtime for kilobytes of RAM, and the arithmetic runs on 8-bit or soft-float cores. Whether a circuit can be simulated on such a device, and whether it is better sent to a PC, is a practical question for embedded and educational quantum tooling. Statevector simulation on a microcontroller has been shown for a few qubits on an RP2040 board [2]; a limited literature search found no peer-reviewed study on Arduino or ESP boards. Dense statevector simulators and their exponential memory demand are well documented [1], as are CPU and GPU simulators such as Qiskit Aer (cited through the Qiskit paper [3]) and fast simulators that trade numerical precision against memory and speed [4]. We claim no novelty over these simulators [1], [3], [4] or the textbook circuits we use (QFT [5], Grover search [6]).

We ask: *what limits statevector simulation on microcontrollers, how well can those limits be predicted, and where is the local-versus-offload trade-off?* Contributions: (i) the memory and time limit of each board and circuit, with the evidence type for each; (ii) a memory rule and two time models with held-out scores; (iii) the offload crossover as brackets of measured n under the fastest and slowest PC simulation speed we observed, so the host PC is part of the result. This is a characterisation study, not a pre-registered test: analysis rules were refined while the data were examined, and every such choice is disclosed in Section IV.

II. System and Method

Simulator and circuits. All boards run the same C++ code: the state is 2^n complex float32 amplitudes (8 B each) and each gate kernel (H, CNOT, controlled-phase CP) sweeps all 2^n indices, so the work of a circuit is (gate calls)$\times 2^n$. The circuits are `h_all` (n H gates), `bell_ghz` (one H, $n-1$ CNOT), `qft` (n H and $n(n-1)/2$ CP) and `grover` ($\lfloor \frac{\pi}{4}\sqrt{2^n} \rfloor$ iterations of a phase oracle and a diffusion step built from recursively decomposed multi-controlled phase gates). The firmware gate sequence was checked against the PC reference gate by gate for $n \leq 7$ (8 for `grover`), and the gate counts against counting kernels for 52 of 52 (circuit, n) pairs. Boards: Arduino Uno R3 (16 MHz, 2 048 B SRAM, static array), NodeMCU ESP8266 (80 MHz, no FPU) and ESP32 DevKit (240 MHz, FPU); one unit each. Boards return timing only, never the final state.

Phases. *Phase 1* compares the float32 build on the PC with Qiskit Aer (infidelity). *Phase 2* runs the circuit on the board for $n = 2, 3, \ldots$ until a memory wall or a per-point time cap (2 h); each point is up to 100 repetitions, reported as the median and interquartile range (IQR) of the `micros()` time around the circuit call. A wall is recorded only with positive evidence (linker RAM overflow on the Uno, failed allocation on the ESP boards). A second ESP32 timestamps a pin raised around the same call (12.5 ns per tick) as an independent but *uncalibrated* reference: a loopback self-test passed 5 of 5 scenarios with a constant offset of 2.0–2.9 µs, but one crystal generates and measures. A counter value is used only if its pairs pass four validity rules, and rule R5 additionally rejects pairs whose counter/`micros()` ratio lies outside $[0.5, 2.0]$ or near the 2^{32}-tick wrap. *Phase 3* offloads: the board sends a 4-byte request (circuit, n, CRC-8) at 115 200 baud, the PC re-simulates with Qiskit Aer on every request, draws one outcome and returns a 6-byte frame; the board times the round trip. It is a latency measurement, not a correctness test. A point with fewer than 30 independent repetitions is *weak*: it is never fitted or scored and is drawn hollow. Several points were measured in more than one session; one selector fixes which session is primary evidence, and the others feed only the reproducibility check. The selector was fixed late, after the data and the first scores were known. Firmware versions are classified by their effect on the timed region (class A: reference; class B: timing-relevant difference, as for all selected ESP8266 `grover` points, which ran a timer interrupt inside the timed region);

points of class B are not fitted.

Memory rule. The state needs $M(n) = 8 \cdot 2^n$ bytes and fits if $M(n) \le B$:

$$n_{\max} = \lfloor \log_2(B/8) \rfloor, \tag{1}$$

with B read from low-n runs only: the SRAM left after the compiler-reported static use (Uno), the free heap at $n = 2$ (ESP8266) and, on the ESP32, the largest free block, since one contiguous allocation is needed.

Time models. With $G_c(n)$ gate calls of circuit c ($G_{\texttt{h_all}} = n$, $G_{\texttt{bell_ghz}} = n$, $G_{\texttt{qft}} = n + \frac{n(n-1)}{2}$) and $N_{c,k}$ calls of kind $k \in \{\mathrm{H}, \mathrm{CNOT}, \mathrm{CP}\}$,

$$\text{M1: } T_{\text{local}}(n) = c_b \, G_c(n) \, 2^n, \tag{2}$$

$$\text{M2: } T_{\text{local}}(n) = \textstyle\sum_k c_{b,k} N_{c,k}(n) \, 2^n + a_b \, G_c(n), \tag{3}$$

with one constant c_b per board (M1), or one non-negative cost per gate kind plus a per-call term, shared by the circuits of a board (M2; non-negative least squares on relative error). Per series the largest three n are held out, every other point with a median of at least 100 µs is fitted unless it is weak or class B, and the score is the mean absolute percentage error

$$\text{MAPE} = \frac{100}{N} \sum_{i=1}^{N} \frac{|T_i^{\text{pred}} - T_i^{\text{meas}}|}{T_i^{\text{meas}}} \tag{4}$$

over the N non-weak held-out points. M1c fits one constant per board and circuit; M2g is M2 fitted without `grover`. A fit is called identifiable if it has at least (parameters +2) points, full column rank and no coefficient on the non-negativity boundary. For `grover`, $G_{\texttt{grover}} = n + k(n)\big(16 \cdot 3^{n-3} + 8n - 6\big)$ for $n \ge 3$, with $k(n) = \max(\lfloor \frac{\pi}{4}\sqrt{2^n} \rfloor, 1)$ evaluated in float32 as in the firmware, and the reference marked-state probability is $P_{\text{marked}} = \sin^2\big((2k+1)\theta\big)$ with $\theta = \arcsin 2^{-n/2}$.

Offload and crossover. The round trip of one request is

$$T_{\text{off}}(n) = T_{\text{wire}} + T_{\text{res},b}(n) + T_{\text{pc},c}(n), \tag{5}$$

the serial wire time $T_{\text{wire}} = 10\,(4 + 6)/\text{baud}$ (0.87 ms), a board-and-link residual (board compute, USB-serial latency, host scheduling; per point, the round-trip median minus the simulation median minus the wire time) and the PC simulation time (Aer build, run and sample, logged per point). T_{pc} is a range: the fastest and slowest value observed over all Phase 3 sessions give a "fast" and a "slow" PC bound. The crossover

$$n^* = \min\{n : \; T_{\text{off}}(m) < T_{\text{local}}(m) \;\; \forall\, m \ge n \text{ measured}\} \tag{6}$$

is reported as a bracket of two consecutive measured n for each bound, without interpolation.

III. Results

Float32 accuracy (Phase 1, PC build, Fig. 1a). `h_all`, `bell_ghz` and `qft` pass the 10^{-3} infidelity gate at every tested n (largest infidelity 2.43×10^{-7}, 3.45×10^{-8}, 2.43×10^{-7}). The raw infidelity of `grover` passes up to $n = 8$ and fails from $n = 9$; this is a loss of norm, not a wrong state: the infidelity of the normalised state stays below 5.5×10^{-5} for $n \le 12$. The float32 constant $1/\sqrt{2}$ (relative error 1.71×10^{-8}) accounts for the loss but over-predicts it by about 16 %. The normalised criterion was chosen after this was understood. The gate itself is the project's own acceptance value, set before the Phase 1 data were collected; it sits well above the float32 rounding of about 10^{-7} per operation, and only the last passing n of `grover` moves when it is changed. A float64 build and Aer both match the closed-form `grover` state ($\le 8.5 \times 10^{-9}$ and $\le 6.2 \times 10^{-11}$). Whether the board builds lose norm likewise was not tested.

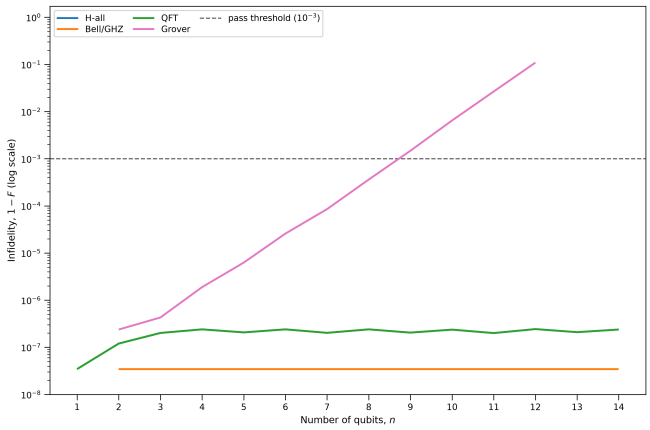
TABLE I
Predicted versus measured memory wall for `h_all`, `bell_ghz`, `qft` (B in bytes, read at $n = 2$). "wrong": the prediction exceeds the last working n.

Board	B from	B (B)	Pred. $n_{\max}$	Last ok	First fail	Evidence
Uno R3	static SRAM	1 842–1 848	7	7	8	build_overflow
ESP8266	total free heap	52 240	12	12	13	malloc_failed
ESP32	largest block	110 580	13	13	14	malloc_failed
ESP32	total free heap	256 328	14 (wrong)	13	14	malloc_failed

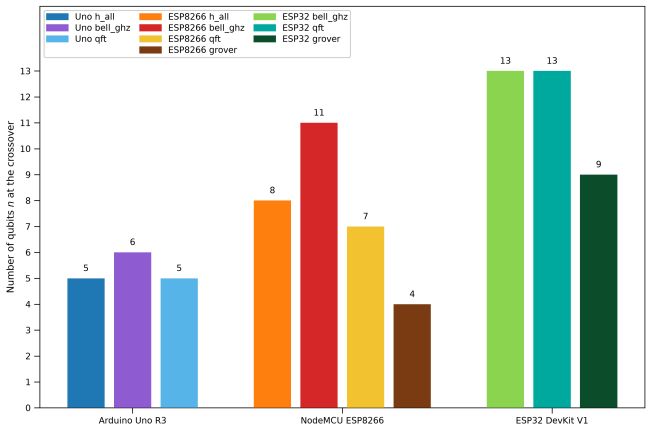
Memory walls. For `h_all`, `bell_ghz` and `qft` the largest working n is 7 (Uno), 12 (ESP8266) and 13 (ESP32), and the first failure is one qubit higher on every board (Table I). Eq. (1) reproduces all 9 board–circuit rows when B is the Uno static RAM, the ESP8266 free heap and the ESP32 *largest free block* (110 580 B); the ESP32 total free heap (256 328 B) predicts $n = 14$, one qubit too many, because the failing state of $n = 14$ needs one contiguous block of 131 072 B. The test resolves B only to a factor of two, the Uno B comes from the same compiler figure that triggers the overflow (close to circular), and the ESP32 predictor was chosen after the walls were known. `grover` on the ESP boards stopped at the time cap (ESP8266 at $n = 10$, ESP32 at $n = 12$) before any wall; their predicted limits (12, 13) are untested.

Execution time. On a log axis the non-`grover` series are roughly straight (Fig. 2), consistent with work $\propto$ gate calls $\times 2^n$. The held-out scores are: M2 reaches 17 % MAPE over 32 non-weak points against 47 % for M1, and a constant per board and circuit (M1c) 28 %; per board M1 scores 45, 67 and 26 % (Uno, ESP8266, ESP32). Clean (firmware class A only) scores are 17 % (M2) and 46 % (M1). The model is not identifiable on the Uno at the primary split (the verdict changes with the held-out size, as a coefficient sits on the non-negativity boundary), ESP8266 `grover` is excluded because its firmware ran a timer interrupt inside the timed region, and M2 was not defined blind with respect to `grover`. M2 scores 25 % (Uno), 17 % (ESP8266) and 6 % (ESP32); without `grover` in the fit (M2g) it reaches 14 %, and the largest error of a single series is Uno `qft` (71 %). Over held-out sizes 2–4 M2 stays within 14.1–17.9 % (M1: 46.4–48.2 %). M2's constants are fits, not gate costs: the fitted ESP8266 CNOT cost corresponds to 4.18 cycles per loop iteration, which is physically impossible.

Timer agreement (Fig. 3). On usable counter points the median `micros()`/counter ratio is 0.9943 (Uno), 0.9974 (ESP8266) and 0.9995 (ESP32); the ESP32 worst point is 0.85 % off. The Uno reads 0.6 % below the counter, which we interpret, without proof, as a property of the Uno timebase.



(a) Phase 1: float32 accuracy (PC)



(b) Phase 4: crossover

Fig. 1. (a) Phase 1, PC build: infidelity $1-F$ of the float32 simulator against Qiskit Aer versus n, without renormalisation, with the 10^{-3} acceptance gate dashed; `h_all` is identical to `qft` and drawn beneath it. (b) Phase 4: the number of qubits at which the Phase 2 and Phase 3 lines cross, rounded up to the first whole n at which offload is faster. It is read from the reference-counter medians with straight segments between measured n, so it is not the bracket of Table II, which uses `micros()` and the round trip. Circuits whose lines do not cross (Uno `grover`, ESP32 `h_all`) are not drawn.

Of the Phase 2 counter points, 19 of 24 (Uno), 29 of 42 (ESP8266) and 44 of 47 (ESP32) pass the validity rules; the counter could not pair most long ESP8266 points nor ESP32 `grover` at $n = 12$ (suspected line noise; unverified). This is agreement between two uncalibrated clocks, conditional on the coarse rule R5 added after the data were seen, and it is not documented above 2.88 s on the ESP boards.

Offload crossover. The round trip has a floor of 3.4 ms (Uno), 3.7 ms (ESP8266) and 6.8 ms (ESP32) at $n \leq 4$, so offload loses at small n and wins once the exponential local time passes it (Fig. 4: the round trip is nearly flat except for `grover`, whose PC simulation grows; Fig. 1b and Table II). 10 of 12 board–circuit pairs have a bracket under both bounds; ESP32 `h_all` has none up to $n = 13$ under the slow bound, and Uno `grover` is already faster offloaded at the smallest n under the fast bound. For all four circuits the bracket moves to larger n from Uno to ESP8266 to ESP32: the faster the board, the later offloading pays (with three boards, speed and link residual of 0.35, 0.72 and 3.27 ms cannot be separated). Read from the reference-counter medians instead (Fig. 1b), the lines cross at $n = 5$, 6 and 5 on the Uno (`h_all`, `bell_ghz`, `qft`), 8, 11, 7 and 4 on the ESP8266 (`h_all`, `bell_ghz`, `qft`, `grover`) and 13, 13 and 9 on the ESP32 (`bell_ghz`, `qft`, `grover`), each inside or next to the corresponding bracket of Table II, except Uno `qft` (5 against the bracket 3–4). Between the two PC bounds the bracket moves by at most 1 measured step in the 11 pairs where offload wins under both (identical in 4, one step apart in 7). 10 of the bracket labels are *within noise* and 4 have a weak point in or beyond the bracket. The width of the band is the host PC's speed, which varied by a median 1.53× (up to 2.92×) between sessions, so the crossover is a property of the pair (board, PC). Beyond the wall only offload exists (Uno to $n = 8$, ESP8266 to 13, ESP32 to 14). Using M2 in Eq. (6), the predicted bracket equals the measured one in 13 of 14 comparisons where M2 is identifiable (M1: 7), is one measured step off in 1 and further off in 0; because local time doubles per qubit while the band is flat, a bracket can match despite a sizeable model error.

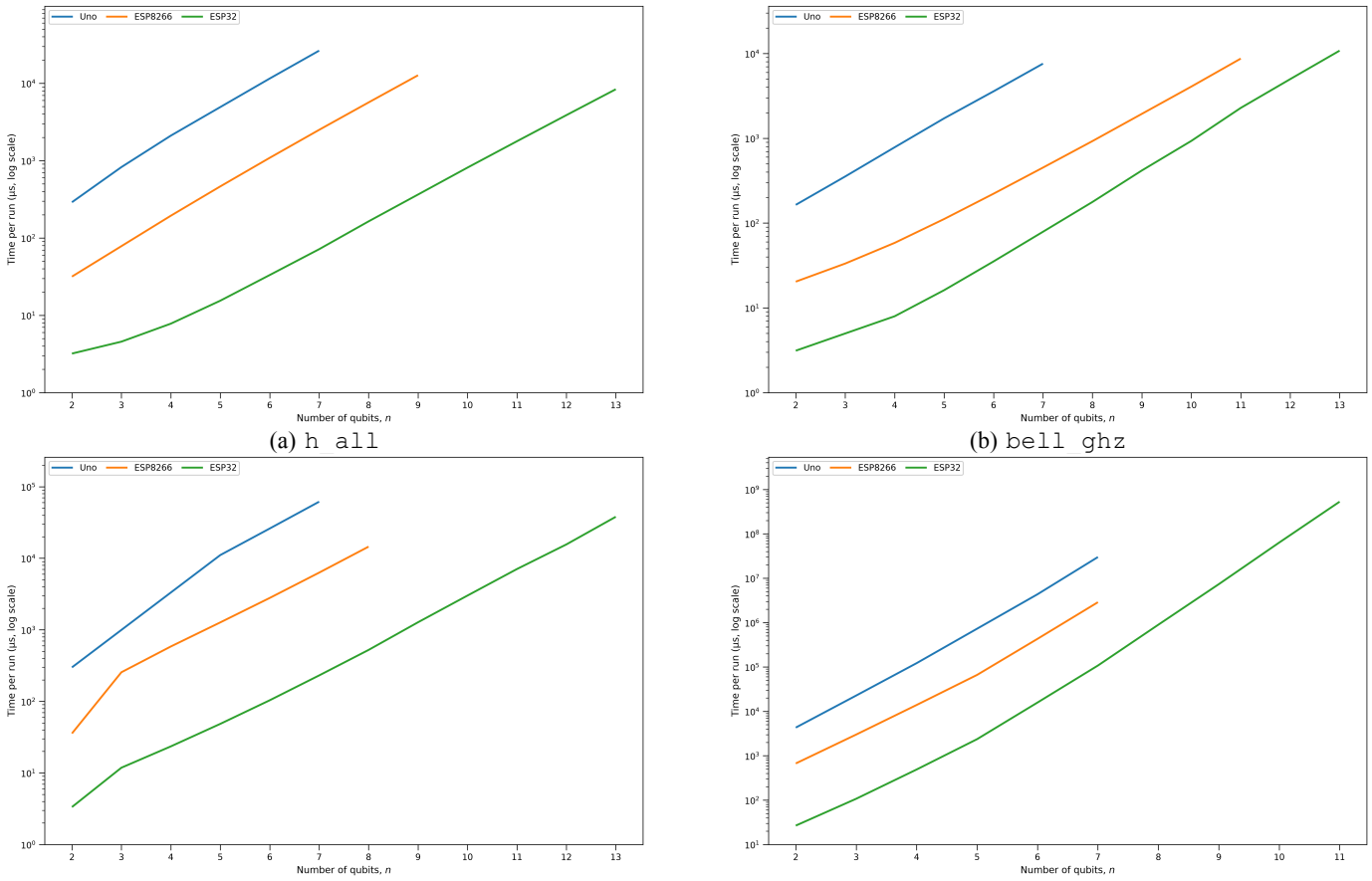
TABLE II
Crossover brackets: two consecutive measured n between which offload becomes faster and stays faster, for the fast and slow PC bound. ≤ 2: faster at the smallest n; none: no crossover. † within noise; ‡ weak point in or beyond the bracket.

Board	Circuit	Measured n	Fast PC	Slow PC
Uno R3	`h_all`	2–7	4–5	4–5
	`bell_ghz`	2–7	5–6	6–7†
	`qft`	2–7	3–4	3–4†
	`grover`	2–7	≤2†	2–3†
ESP8266	`h_all`	2–12	7–8†	7–8
	`bell_ghz`	2–12	9–10†	10–11†
	`qft`	2–12	6–7†	7–8
	`grover`	2–10	3–4‡	4–5‡
ESP32	`h_all`	2–13	12–13†	none†
	`bell_ghz`	2–13	12–13	12–13
	`qft`	2–13	11–12	12–13
	`grover`	2–12	8–9‡	9–10‡

Reproducibility. Phase 2 timer medians of 9 points measured in more than one session (all `grover` on the ESP boards) differ by at most 0.0056 %. Phase 3 round trips of 45 ESP8266 points differ between the original and the repeat session by a median 14 % (maximum 114 %), mostly through the PC simulation time (median 21 %), which the band of Table II covers; the cause of the PC speed change was not recorded.

IV. Discussion and Limitations

What limits each board. On the Uno the limit is the 2 048 B SRAM: the state of $n = 8$ no longer fits next to 200–206 B of static use. On the ESP8266 it is the free heap and on the ESP32 the largest contiguous block. Memory needs one low-n read; time needs a model, and per-gate-kind costs lower the held-out error from 47 % to 17 % but remain fits. Offload has a floor of a few milliseconds (serial link, USB



(a) `h all` (b) `bell ghz`

(c) `qft` (d) `grover`

Fig. 2. Phase 2: local execution time per run (reference-counter median, μs, log scale) versus n for the Uno, ESP8266 and ESP32 (blue, orange, green), one panel per circuit. Only points whose counter pair passed the validity rules are drawn, joined by straight segments, so a line can skip an n.

latency, host scheduling and the Aer build and run; the split was not tested), so local execution on a fast board wins below the crossover and is impossible beyond the wall.

Scope. One unit per board, one PC and one baud rate were measured; how the brackets move with another PC or link is not tested. There are no board-level correctness data, and Phase 3 compares its single sampled outcome with nothing. No hash of the flashed binary was recorded, and firmware versions were classified from the source diff.

Measurement. The counter is uncalibrated and its offset of 2.0–2.9 µs cannot be split between generator and counter. The Uno timer resolves 4 µs, the ESP timers 1 µs. In Eq. (5) the residual absorbs the error of subtracting two medians, so it is bookkeeping, not a measurement. The ESP8266 Phase 3 data come from one repeat session, and two ESP32 `grover` points contain repeated rows (15 and 29 independent of 100 recorded rows), of which only the independent ones are used.

Post-hoc choices. The following were set after data or scores were seen: the weak-point rule, the firmware classes, the identifiability rule, the primary Phase 1 criterion, the selection of the session that supplies each point, counter rule R5, the choice of the memory predictor per board, the band-and-bracket design of the crossover and the predicted-crossover comparison. The split (largest three n) and the 100 µs floor were fixed before scoring by the authors' record, which cannot be verified from the commit history. The effect of the rules together was not split.

V. Conclusion

Memory ends `h_all`, `bell_ghz` and `qft` at $n = 8$, 13 and 14 on the Uno, ESP8266 and ESP32, while `grover` on the ESP boards stopped at the time cap. The wall is consistent with one low-n memory read, held-out time is predicted to about 17 % with a model that is a fit, not a gate cost, and offload overtakes local execution within brackets of measured n that depend on the host PC. All numbers and figures are regenerated from the raw CSVs by one command; the repository link is withheld for blind review.

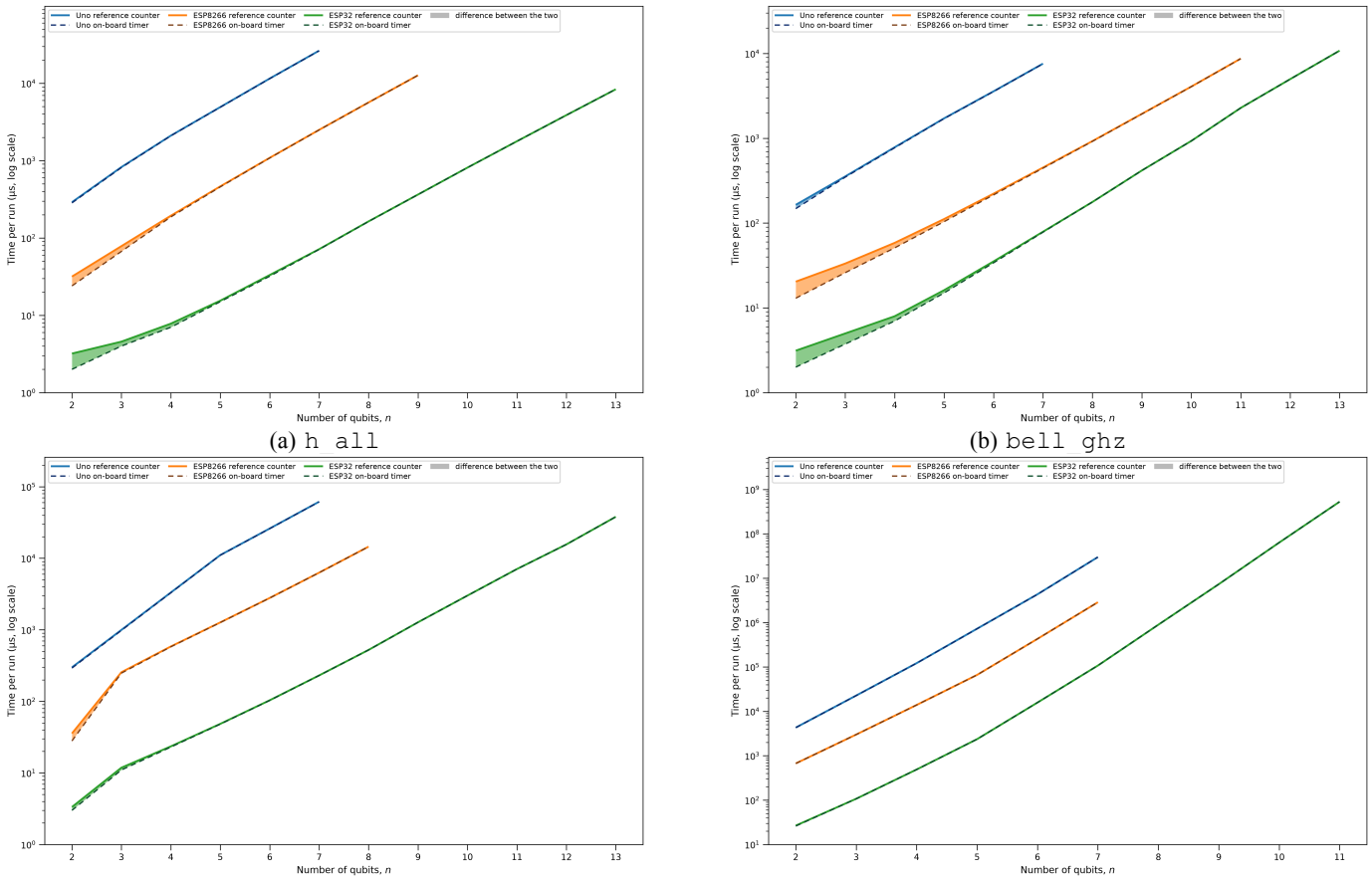
References

[1] T. Häner and D. S. Steiger, "0.5 petabyte simulation of a 45-qubit quantum circuit," in *Proc. Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC '17)*, 2017, p. 33, arXiv:1704.01127.

[2] M. Carney, "Quid manumit – freeing the qubit for art," arXiv:2309.03104; ISQCMC, Berlin, 2023.

[3] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," arXiv:2405.08810, 2024.

[4] S. Yusufov, C. Stefanski, and C. Gonciulea, "Designing a fast and flexible quantum state simulator," arXiv:2303.01493, 2023.



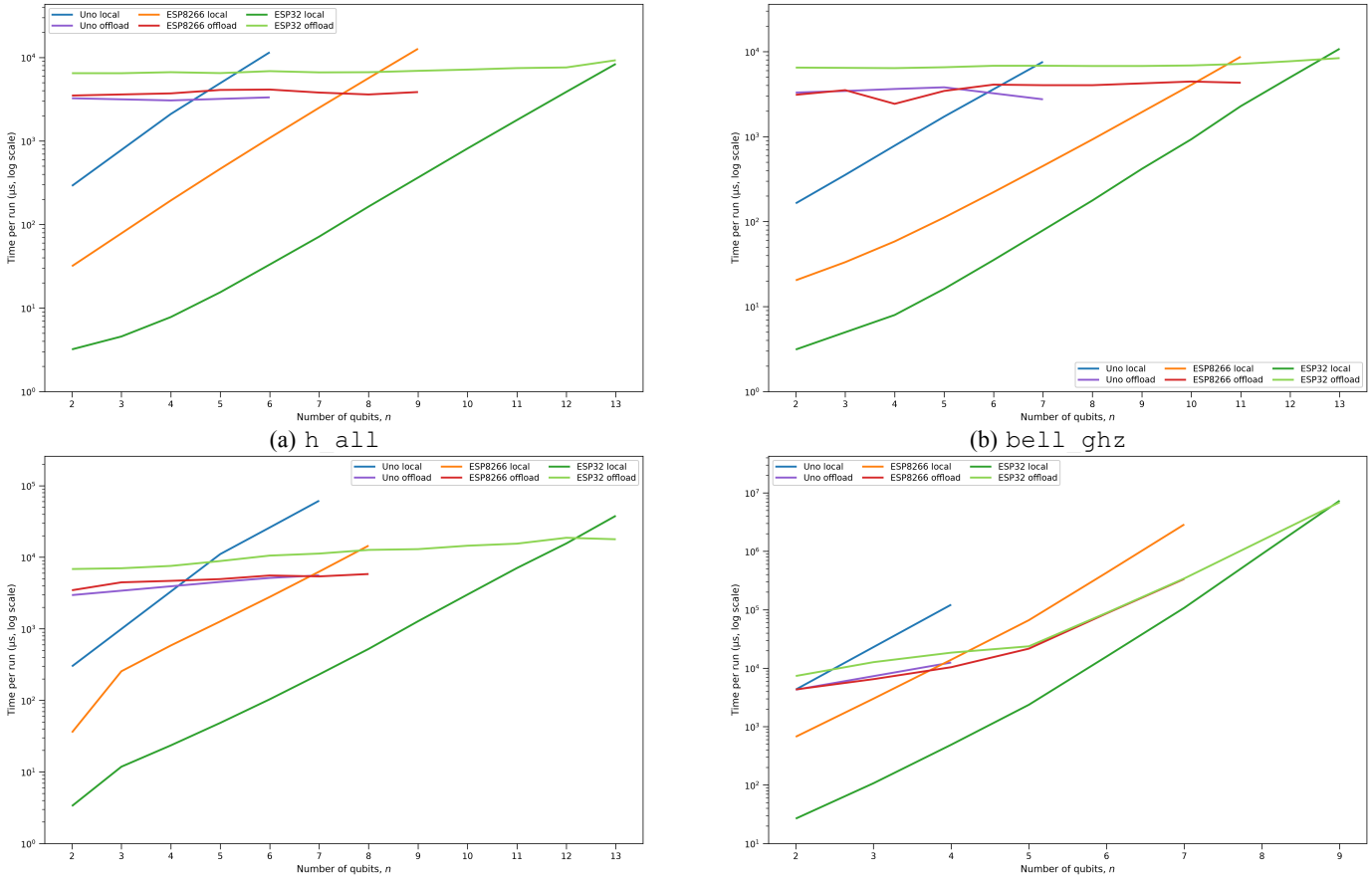
(a) `h all` (b) `bell ghz`

(c) `qft` (d) `grover`

Fig. 3. Phase 2: reference counter against the on-board timer, time per run (μs, log scale) versus n, one panel per circuit, Uno, ESP8266 and ESP32 in blue, orange and green. Solid: reference counter (a separate ESP32); dashed: the board's own `micros()`; shaded: the difference between the two.

[5] D. Coppersmith, "An approximate Fourier transform useful in quantum factoring," IBM Research Division, Tech. Rep. RC 19642, 1994, arXiv:quant-ph/0201067 (2002).

[6] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annual ACM Symposium on the Theory of Computing (STOC)*, 1996, pp. 212–219, arXiv:quant-ph/9605043.



(a) `h all` (b) `bell ghz`

(c) `qft` (d) `grover`

Fig. 4. Phase 4: local (Phase 2) and offload (Phase 3) time per run versus n on the same axes, one panel per circuit, from the reference-counter medians. The point where an offload line crosses the local line of the same board is the crossover of Fig. 1b.